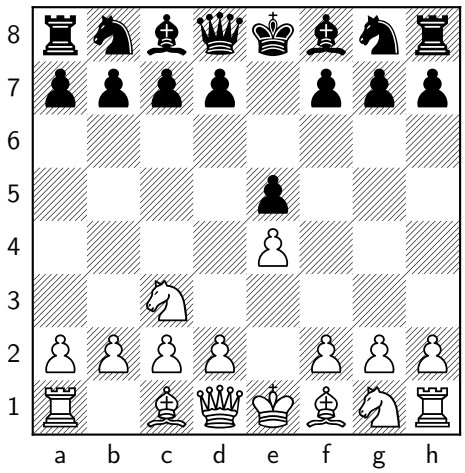
Project Assignment

105.173 Bayesian Statistics



The Vienna Game

Introduction

In competitive games, determining which player is superior can be challenging, particularly when using an overall scoring system to rank players against one another in a tiered list. This issue is not limited to human competitors; For instance, when deciding which chess engine to favor in an upcoming competition, we encounter a scenario relevant to software engineers developing these programs. One of the most well-known chess engines is Stockfish, which is also the reigning champion[1].

A common decision encountered during the development of a chess engine is whether modifications to the engine's internal mechanisms have genuinely enhanced its competitiveness.[2] Basically the only method for assessing improvements is to have the previous version compete in a series of games against the new version. Based on the results, developers can determine if the changes have positively impacted the engine's performance. However, if the chess engine is not already very strong, this process can be misleading, as the new version may perform well only against its predecessor but not necessarily demonstrate overall superiority. To address this concern, it is beneficial to include other engines of similar strength in the tournament, allowing for a more comprehensive evaluation of the changes made to the program.

Another common issue is that the underlying algorithms of a chess program depend on many parameters. Finding a good set of parameters is a monumental task, especially in this context, as many parameters have unintuitive interactions with each other. There are different approaches for "tuning" the engine parameters (not altering its programming), which is basically an optimization problem. The main issue here is that the optimization objective cannot be directly evaluated but must be approximated in a similar way as before through self-play or tournaments between different parameterizations. This is a very time-consuming process as those tournaments themselves take time.

This is where this project comes in. We have our own engine, called `SchachMaus`. The goal of this project is to use Bayesian approaches to improve `SchachMaus`.

The first step is to establish Bayesian models for analyzing the relative performance of `SchachMaus` in

[1] Top Chess Engine Championship, Computer Chess Rating Lists
[2] Stockfish uses its own distributed testing framework tests.stockfishchess

relation to both its previous versions and other competing engines.

We will then proceed in the direction of Bayesian optimization based on the previously derived performance models to tune the parameters of `SchachMaus`. This approach aims to build an automated tuning procedure that updates our beliefs about which parameter combinations are most effective based on empirical results. This allows us to test parameter settings that are, on the one hand, believed to improve performance, or, on the other hand, possess high uncertainty, rather than relying on manual tuning or exhaustive trial and error.

Comment: This document is self-contained in the context of this course assignment, and all external or chess related references are provided for supplementary information and contextual background. You do *not* need to be familiar with these materials to complete the assignment. The content of this document, referenced software (`R`, including Stan `R` binding packages and the provided `Rchess` package) as well as any provided data is sufficient for the task at hand.

1 Tournament Ranking

(Private Group 2 – 11.11.2026)

The initial phase of this assignment is to analyze a collection of games played between our engine[3] and several competitors on lichess.org. The objective is to construct a rating list for all participants.

1.1 Data

The data is provided as a CSV file named `games.csv` available on TUWEL. This file contains four columns:

`timecontrol` : Specifies the time available for a game, given in the "common" format `time+increment`, where `time` represents the initial available time and `increment` is the additional time added to each engine's clock after every move, measured in seconds. If an engine's time reaches zero, it results in sudden death. For example, a time control of `30+1` means each player starts with 30 second and receives an additional 1 second after each move.

`white` : The name (identifier[4]) of the engine playing as white.

`black` : The name (identifier) of the engine playing as black.

`result` : Outcome of the game, one of three cases; `1-0` (white wins), `1/2-1/2` (draw) and, `0-1` (black wins).

1.2 Modeling

We intend to build a ranking of the engines participating in the games. To achieve this, we will assign each engine a rating. These ratings will be relative to the pool of participants and will be linked to a paired winning probability based on the difference in their ratings.

More specifically, we will develop an "Elo"-like rating system[5] using a Bayesian approach[6]. The classic Elo system calculates the win probability between two opponents with ratings R_A and R_B using the following

[3] Specifically, our engine `SchachMaus` is a striped down version of Shach Hörnchen playing on lichess.org at the end of 2025.
[4] Reference engine names are obscured and labeled A through D.
[5] See: `https://en.wikipedia.org/wiki/Elo_rating_system` [accessed Okt 28, 2025] for more information.
[6] Using `Stan` is not required but strongly recommended.

formula:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}.$$

In this equation, E_A represents the expected score[7] of player A against player B. The expected score is defined as the win rate of player A plus half of the draw rate when competing against player B.

Set up your prior for the rating R_A of any engine A with a mean of 2000, creating an overall pool of ratings centered around this value[8].

Note: `timecontrol` *can be ignored for this part of the project.*

1.3 Results

Using your model, generate a tier list that ranks the engines according to their expected ratings, highlighting instances of superiority where applicable. You are responsible for determining how to assess this.

Comment: The primary intention of this initial stage of the project is twofold. First, we provide a classic setup for Bayesian data analysis, starting from basic data preprocessing (which is almost always a part of any data analysis) through a "simple" model declaration down to a final interpretation. Second, this serves as a foundation for later parts of the project.

2 Tournament Ranking with Time Control

(Private Group 4 – 18.11.2026)

As the available time for "thinking" is a major component of the competitive strength in chess, we now continue with incorporating time control into our ranking model for the analysis of the data from Section 1.1. This means that we intend to have a time control dependent rating of all the competing engines over the range of played time controls.

2.1 Expected Game Duration

To allow comparisons across various time controls, we define the *estimated game duration* as utilized on lichess.org, calculated by the formula $\texttt{time} + 40 \times \texttt{increment}$. This formula assumes an average of 40 moves per side during a game. This method significantly simplifies our analysis by converting the time control into a positive real number t_c, expressed in seconds.

2.2 Modeling

Extend your model from Section 1 to account for the effect of time control. Meaning, we seek to determine a rating $R_A(t_c)$ for each engine A dependent on the time control t_c (the expected game duration).

Note: For the sake of simplicity, you may model the time control dependence linearly.

[7] This is analogous to logistic regression for estimating the expected score based on the rating difference.

[8] Hint: A reasonable prior standard deviation is approximately 200, but it may need to be adjusted based on the requirements of your model.

2.3 Results

Based on the new model, generate a plot of ratings $R_A(t_c)$ over time control t_c for all engines A. The plot should also include a visualization of the ratings uncertainty (for example $\pm$ standard deviation) of the expected rating given time control under your model.

Moreover, similar to the tier list in Section 1.3 build four lists, one for each of the following time controls;

`15+0`	(Ultra Bullet)	`60+0`	(Bullet)
`180+2`	(Blitz)	`600+0`	(Rapid).

3 Tournament Ranking Model Checking

(Open Group 6 – 25.11.2026)

Till now, we only focused on building an initial model. An important part of the modeling process is to verify if our model is capturing the relevant parts of the data under analysis. Using posterior predictive checks we can identify aspects of the data that the model does not capture.

Perform posterior predictive checks for your initial models, first without time control and then with time control. If your models do not pass your posterior predictive checks, alter your models to be consistent with the observations.

3.1 Results

Present the posterior predictive checks for your final models, both with and without time control.

Keep a log on your model iterations with a short but clear reason on what is changed and why (if any.)[9]

[9] No reason for too much detail! More in the direction of "we tested ... to solve ... and landed on ...."